159.302 Games

Approaches to building two player games

n.h.reyes@massey.ac.nz

Reference: ***MIT OpenCourseWare***

Games

Board Games & Search

- **Move generation**
- **Static evaluation**
- **Min-Max**
- **Alpha-Beta**
- **Practical Matters**

1949 Shannon paper

1951 Turing paper

1958 Bernstein paper

55-60 Simon-Newell program (α-β McCarthy?)

66-67 MacHack 6 (MIT AI)

70's NW Chess 4.5

80's Cray Blitz

90's Belle, Hitech, Deep Thought, Deep Blue

Games

Game Tree Search

Initial State: initial board position and player

Operators: one for each legal move

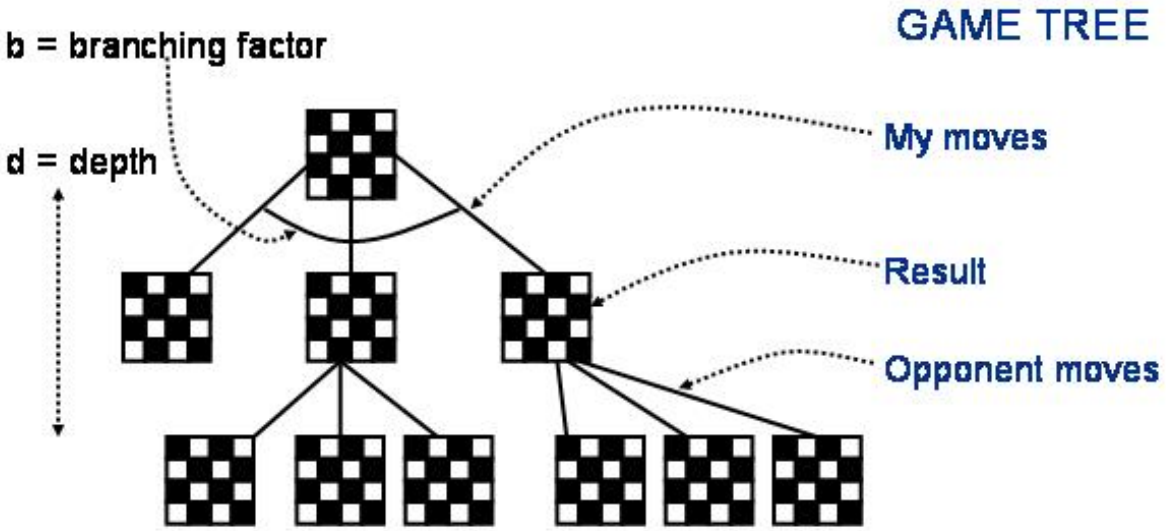
Goal States: winning board positions

Scoring Function: assigns numeric value to states

Game tree: encodes all possible games

- We are not looking for a path, only the **next move** to make (that hopefully leads to a winning position)
- Our best move depends on what the other player does.

Move Generation



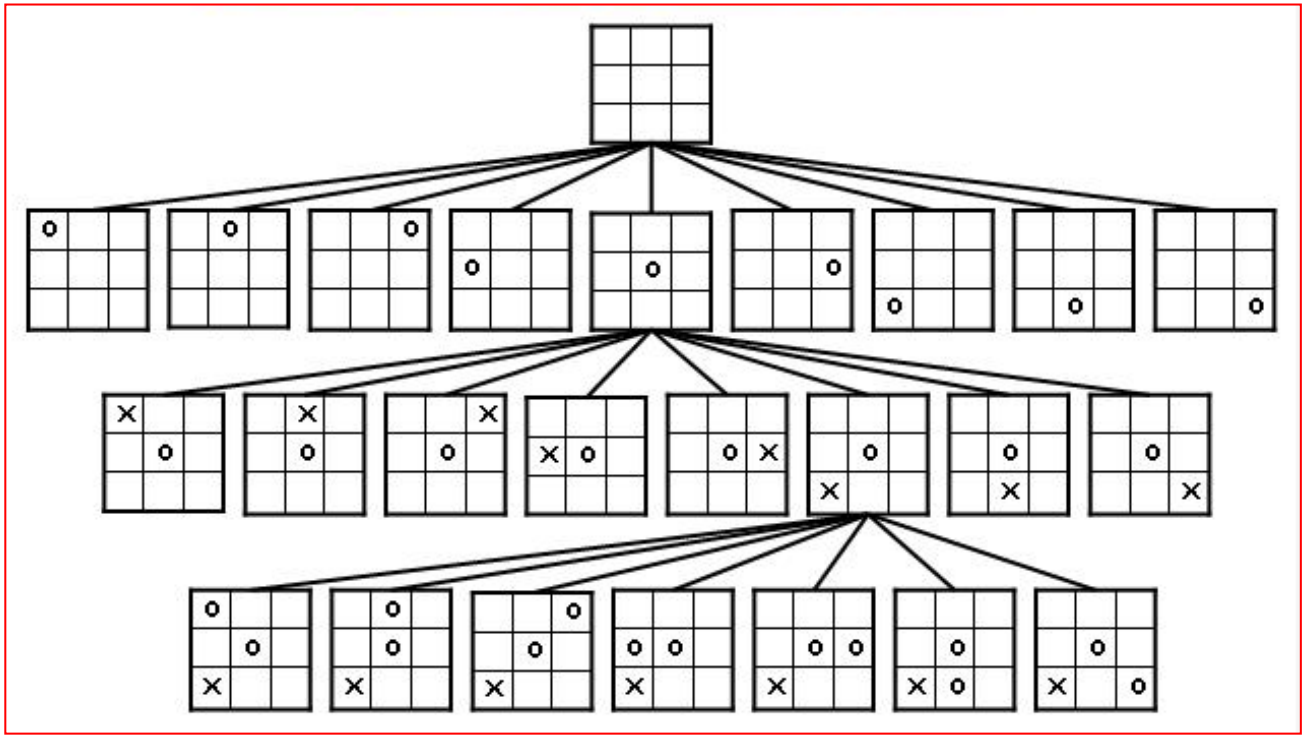
Chess (according to Google DeepMind)

$b \approx 35$

$d \approx 80$ 35^{80} is big!

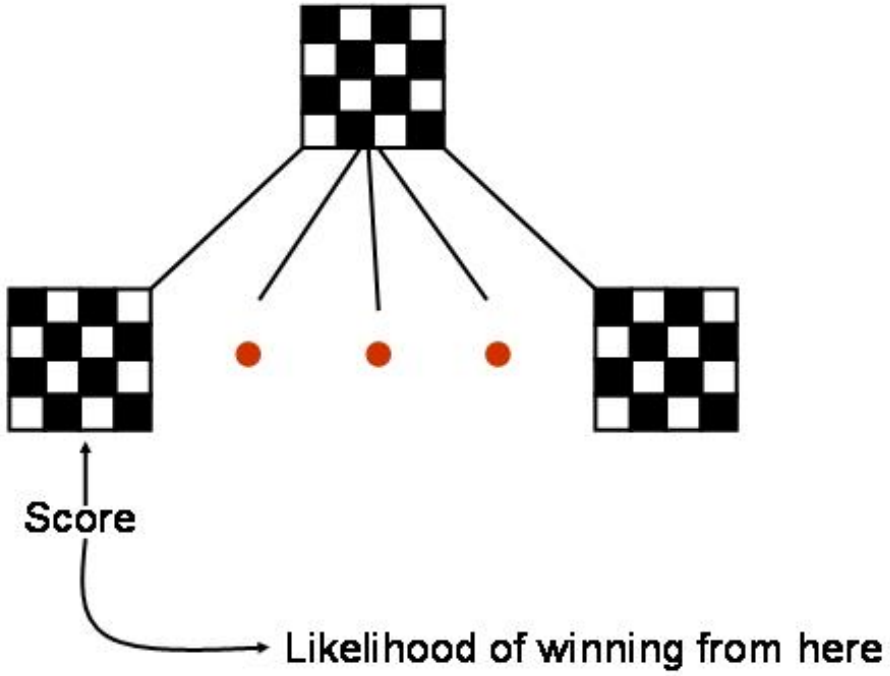
[1]Google DeepMind, 5 New Street Square, London EC4A 3TW, UK. 2Google, 1600 Amphitheatre Parkway, Mountain View, California 94043, USA.

Partial Game Tree for Tic-Tac-Toe



Even for this trivial game, the search tree is quite big.

Scoring Function



Assigns a numerical value to a board position.

Scoring Function: Static Evaluation

A linear function in which some set of coefficients is used to weight a number of "***features***" of the board position.

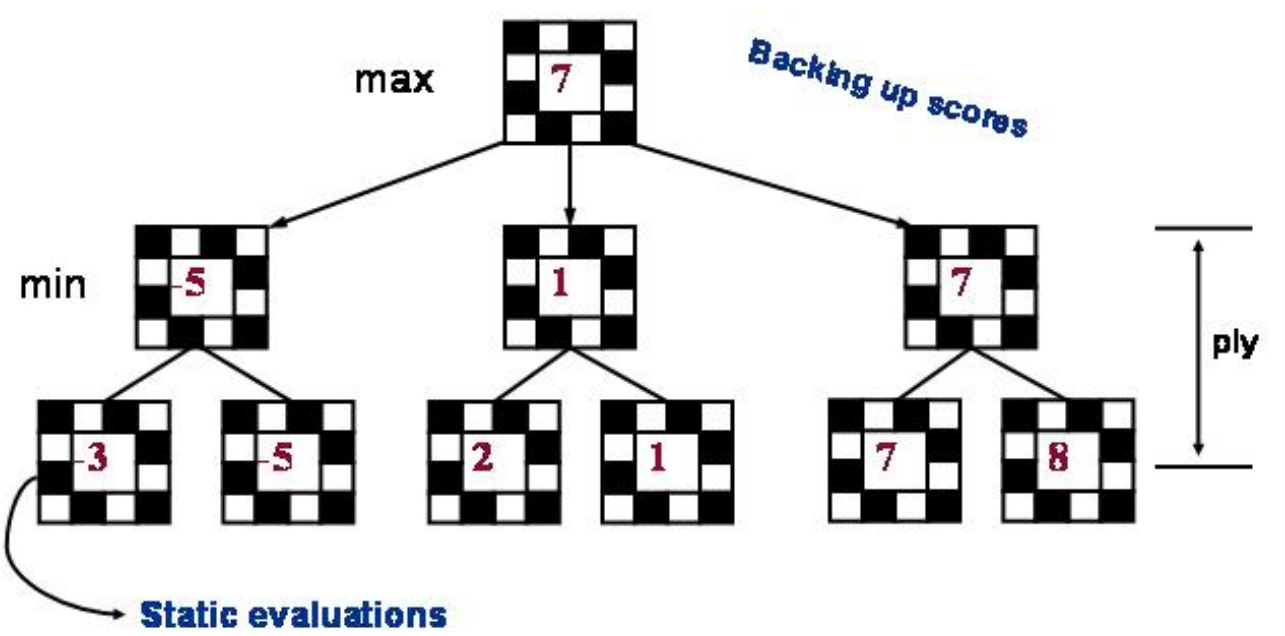
$$S = c_1 \times \text{material} + c_2 \times \text{pawn structure} + c_3 \times \text{mobility} + c_4 \times \text{king safety} + c_5 \times \text{center control} + \ldots$$

P	1
K	3
B	3.5
R	5
Q	9

Too weak to predict ultimate success.

Games

Limited look ahead + Scoring



The tree is generated on the fly.

The static evaluation is performed only on the leaf nodes of the tree.

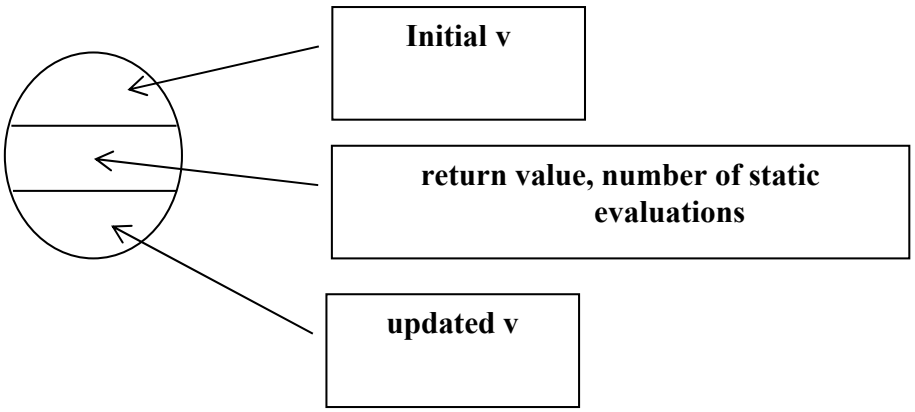
The Min-MaX Algorithm

Min-MaX Algorithm

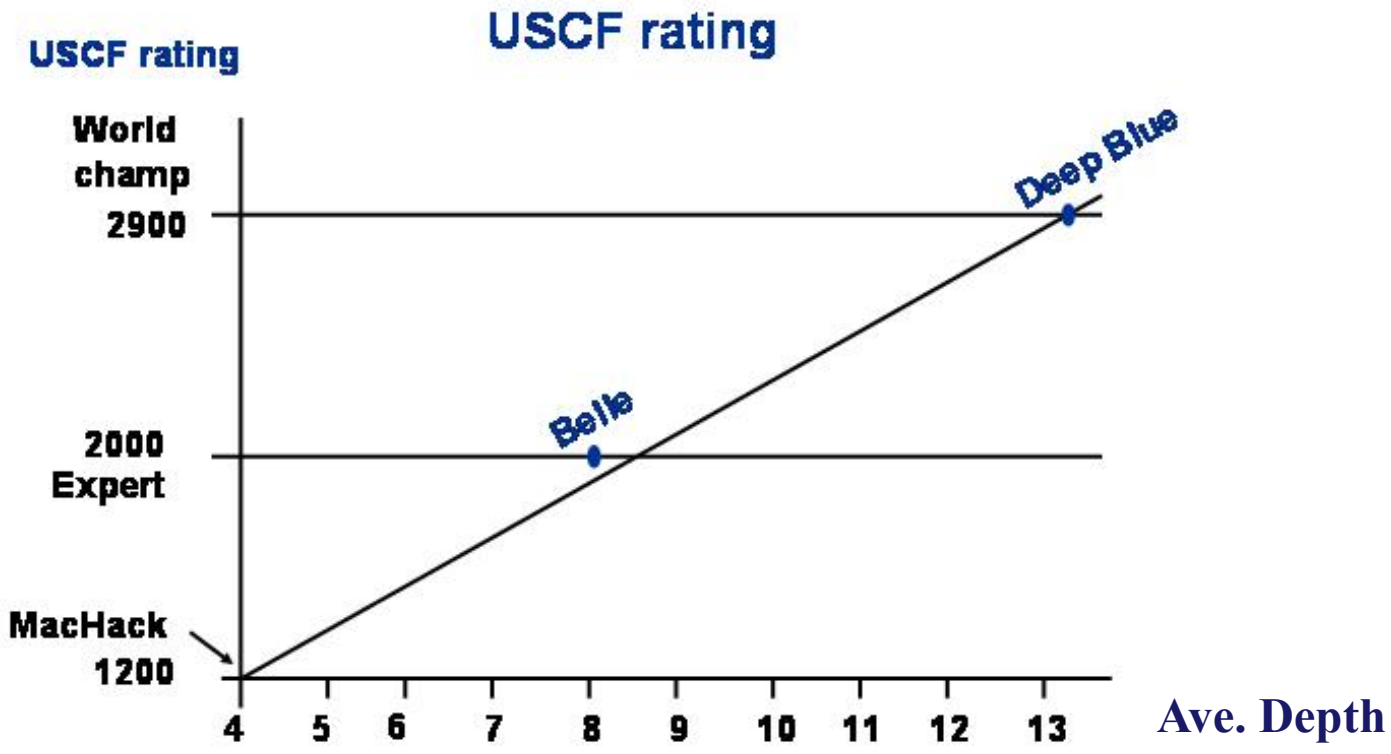
Initial call is ***MAX·VALUE****(state, MAX·DEPTH)*

- *function* ***MAX·VALUE****(state, depth)*
  - *if (depth == 0) then return EVAL(state)*
  - *v = -∞*
  - *For each s in SUCCESSORS(state) do*
    v = MAX(v, ***MIN·VALUE****(s, depth – 1))*
  end
  return v

- *function* ***MIN·VALUE*** *(state, depth)*
  - *if (depth == 0) then return EVAL(state)*
  - *v = ∞*
  - *For each s in SUCCESSORS(state) do*
    v = MIN(v, ***MAX·VALUE****(s, depth – 1))*
  end
  return v


Initial v
return value, number of static evaluations
updated v

USCF Rating



Somehow, it seems as if brute-force search is all that matters.

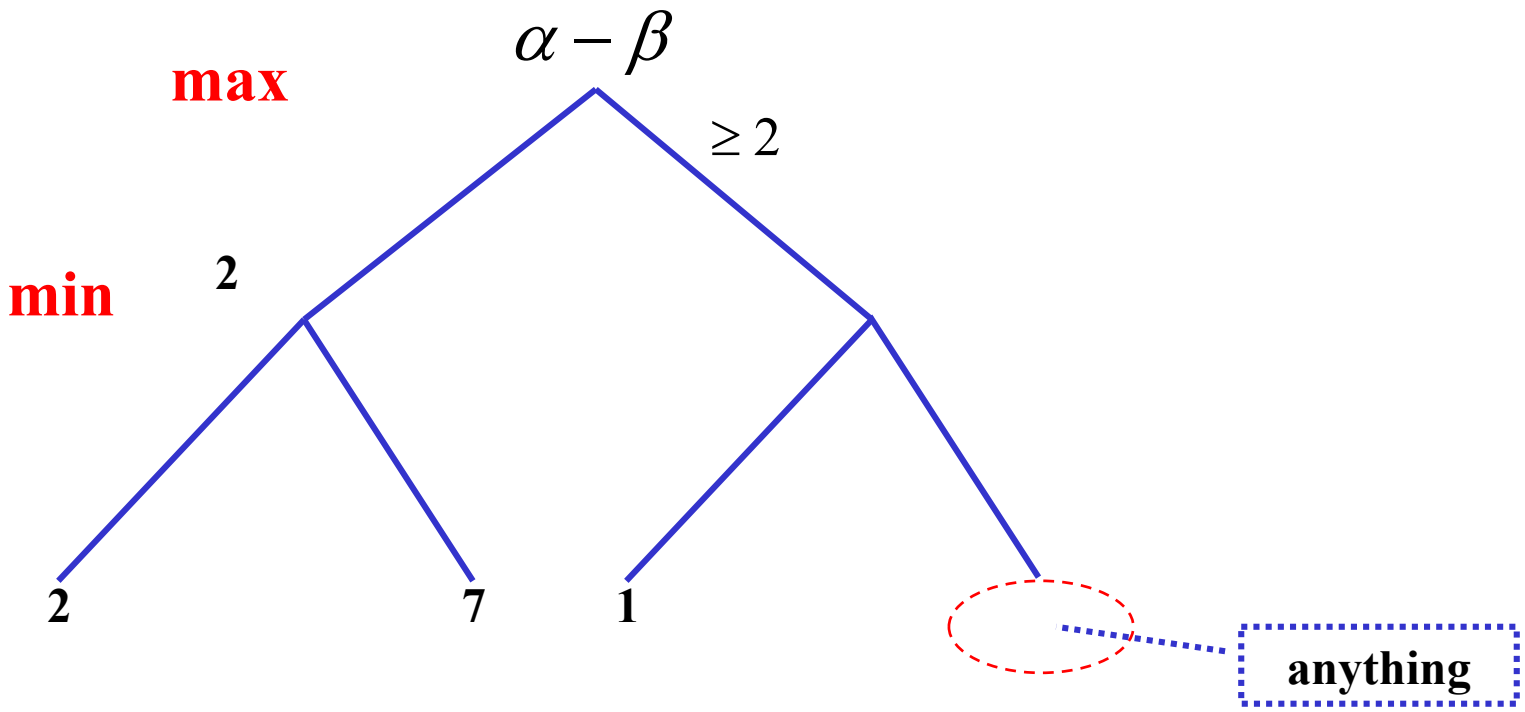
Games

Deep Blue

32 SP2 processors
each with 8 dedicated chess processors
= 256 CP

50-100 billion moves in 3 min
13-30 ply search

Alpha-Beta Pruning



α – is the lower bound on score

β – is the upper bound on score

Alpha-Beta Pruning

α – is the best score for MAX; **β** – is the best score for MIN
Initial call is ***MAX·VALUE****(state, -∞, ∞, MAX·DEPTH)*

function ***MAX·VALUE****(state, α, β, depth)*

- *if (depth == 0) then return EVAL(state)*
- *For each* ***s*** *in SUCCESSORS(state) do*
  α *= MAX(α,* ***MIN·VALUE****(s, α, β, depth-1))*
  If(α ≥ β) Then return ***α*** *//cut-off*

end

return α

Maximises α, returns α

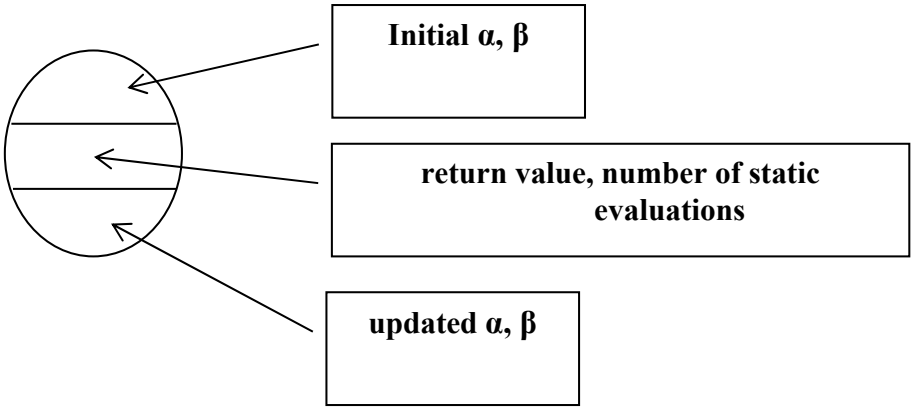
function ***MIN·VALUE****(state, α, β, depth)*

- *if (depth == 0) then return EVAL(state)*
- *For each* ***s*** *in SUCCESSORS(state) do*
  β *= MIN(β,* ***MAX·VALUE****(s, α, β, depth-1))*
  If(β ≤ α) Then return ***β*** *//cut-off*

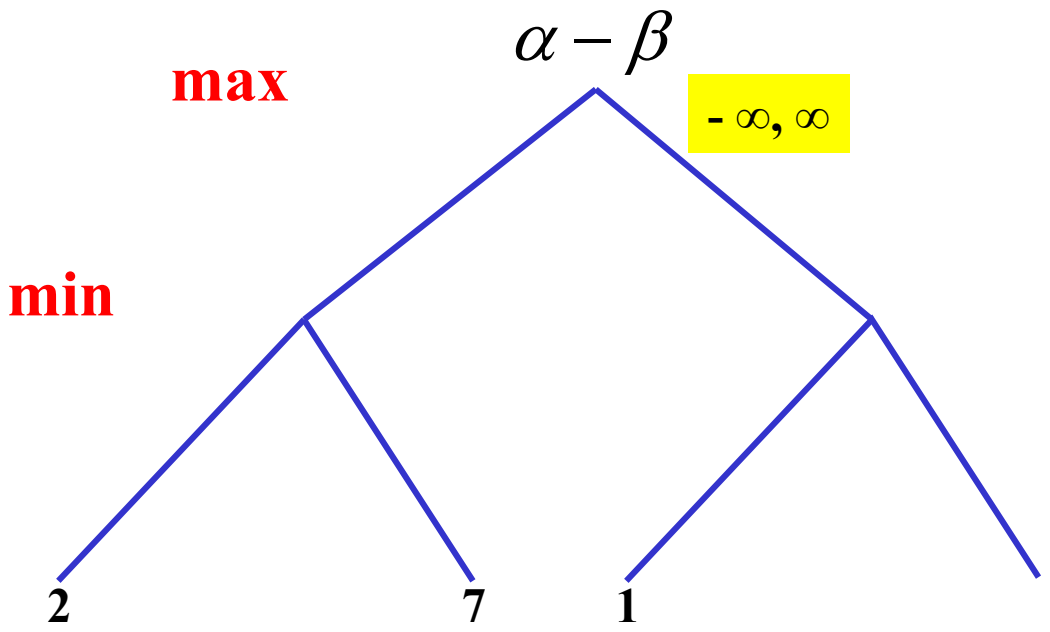
end

return ***β***

Minimises β, returns β


Initial α, β
return value, number of static evaluations
updated α, β

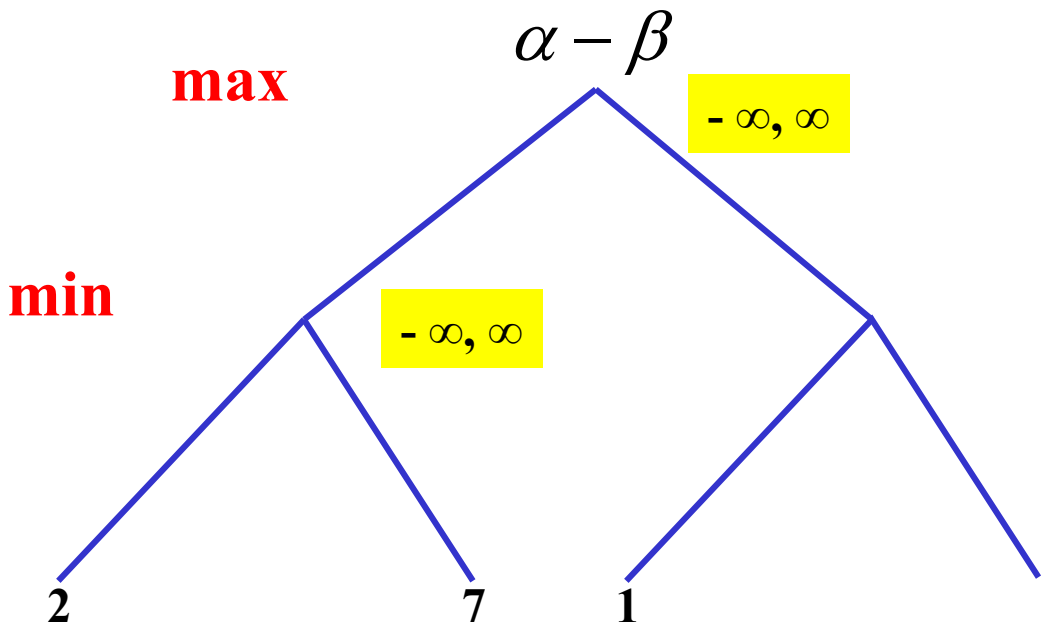
Alpha-Beta Pruning in action



We start with an initial call to ***MAX·VALUE.***

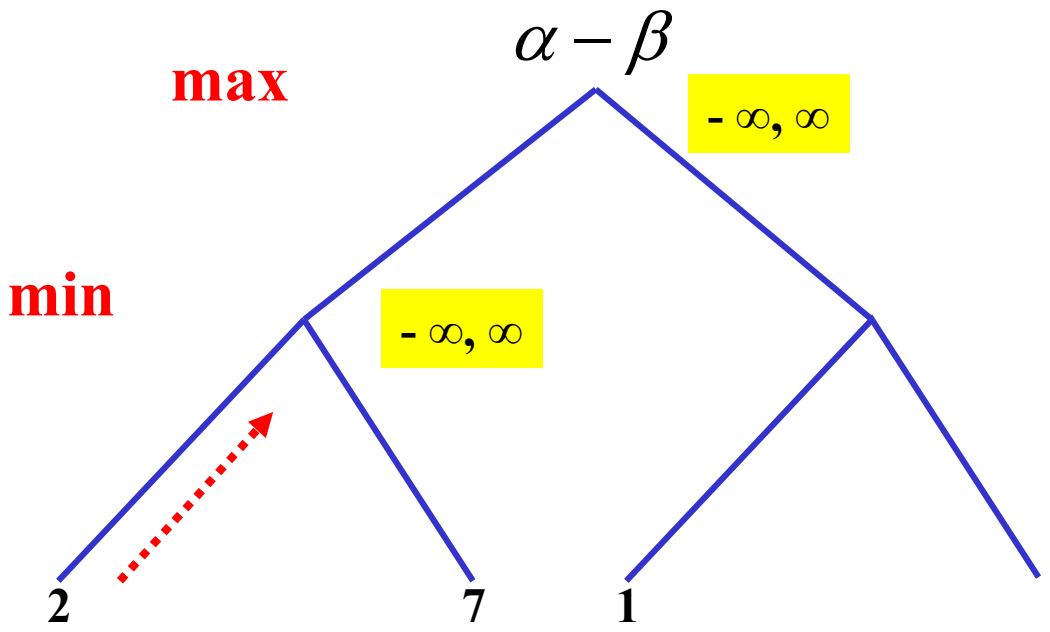
MAX·VALUE*(state, -∞, ∞, MAX·DEPTH)*

Alpha-Beta Pruning in action



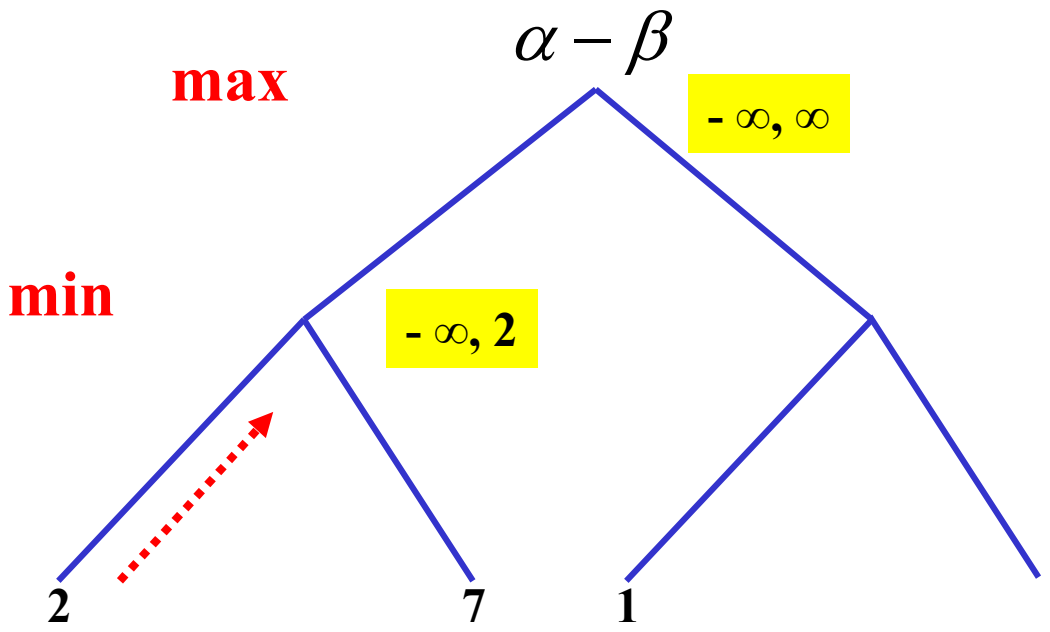
MAX·VALUE *now calls* ***MIN·VALUE*** *on the left successor with the same values of alpha and beta.*
MIN·VALUE *now calls* ***MAX·VALUE*** *on its leftmost succesor.*

Alpha-Beta Pruning in action



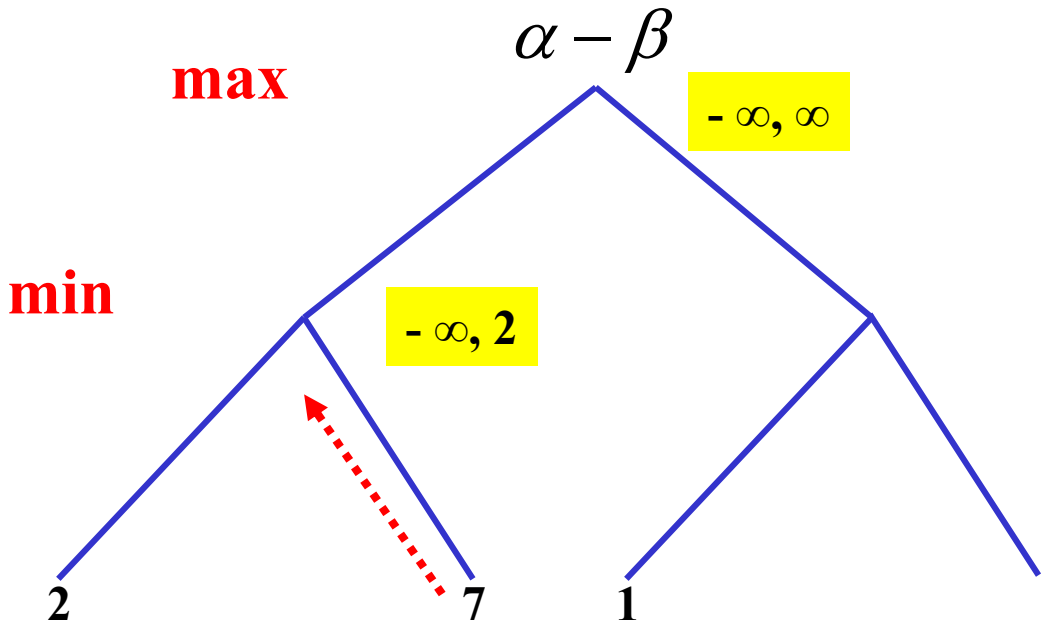
MAX·VALUE *is at the leftmost leaf, whose leaf value is 2 and so it returns that.*

Alpha-Beta Pruning in action



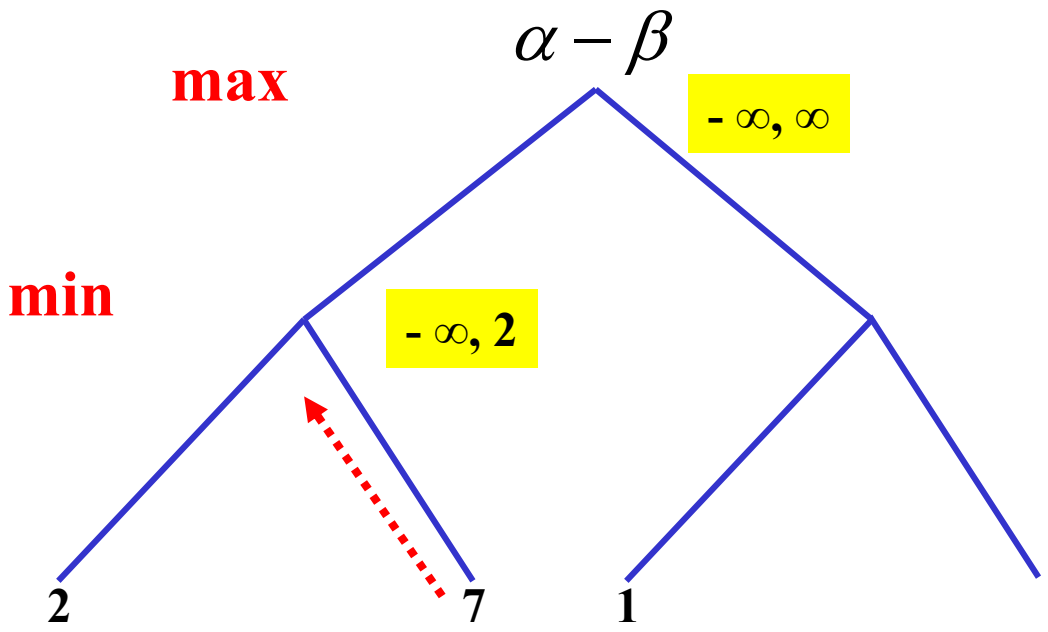
This first value, since it is less than ∞, becomes the new value of ***β*** *in* ***MIN·VALUE.***

Alpha-Beta Pruning in action



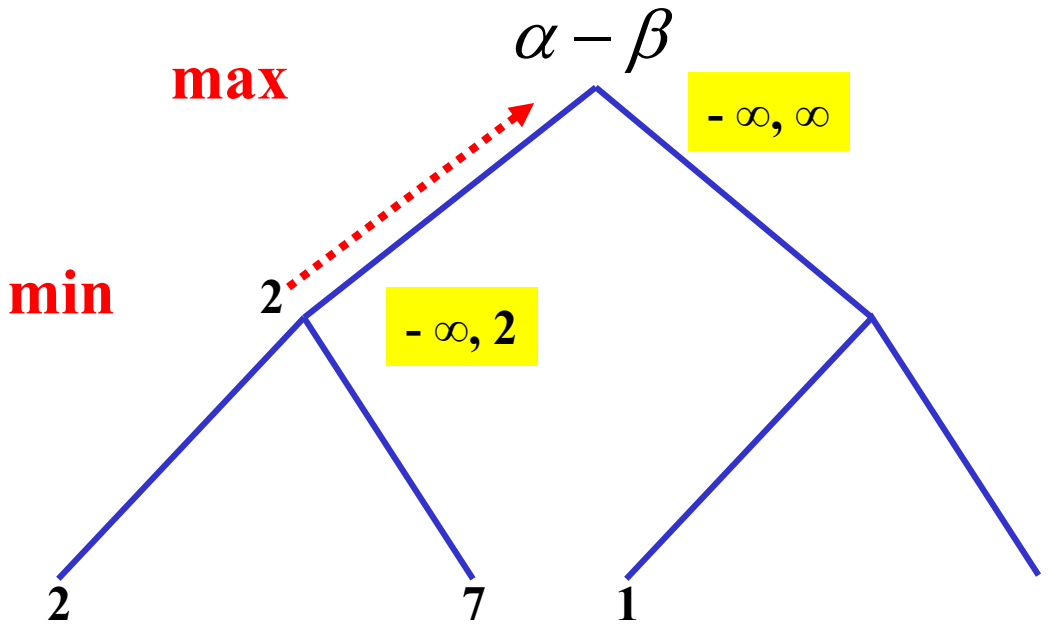
*So now we call **MAX·VALUE** with the next successor, which is also a leaf whose value is 7.*

Alpha-Beta Pruning in action



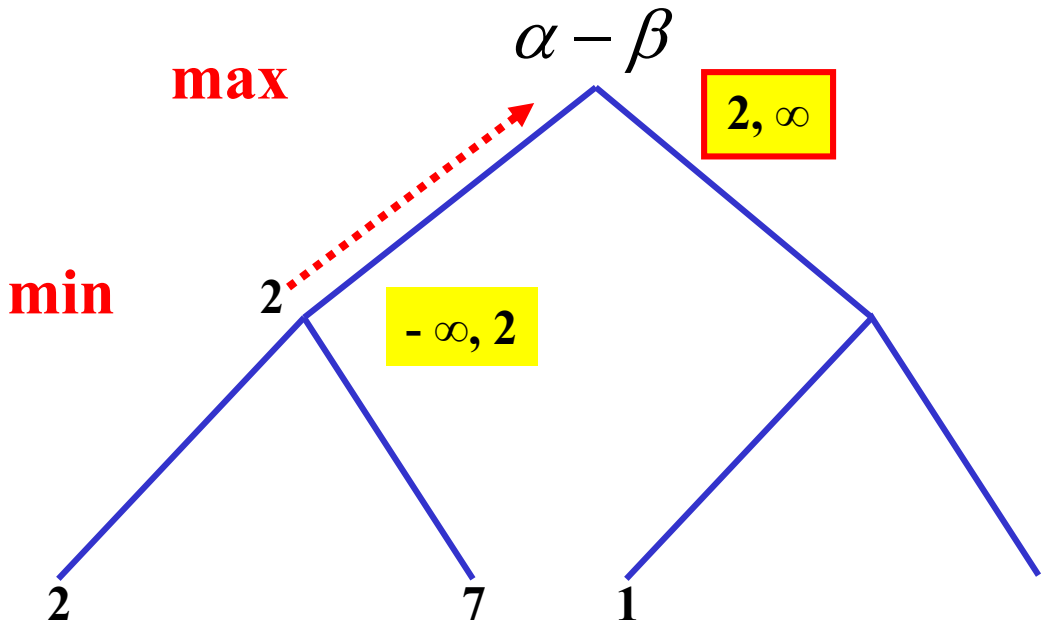
7 is not less than 2 and so the final value of **β** *is 2 for this node.*

Alpha-Beta Pruning in action



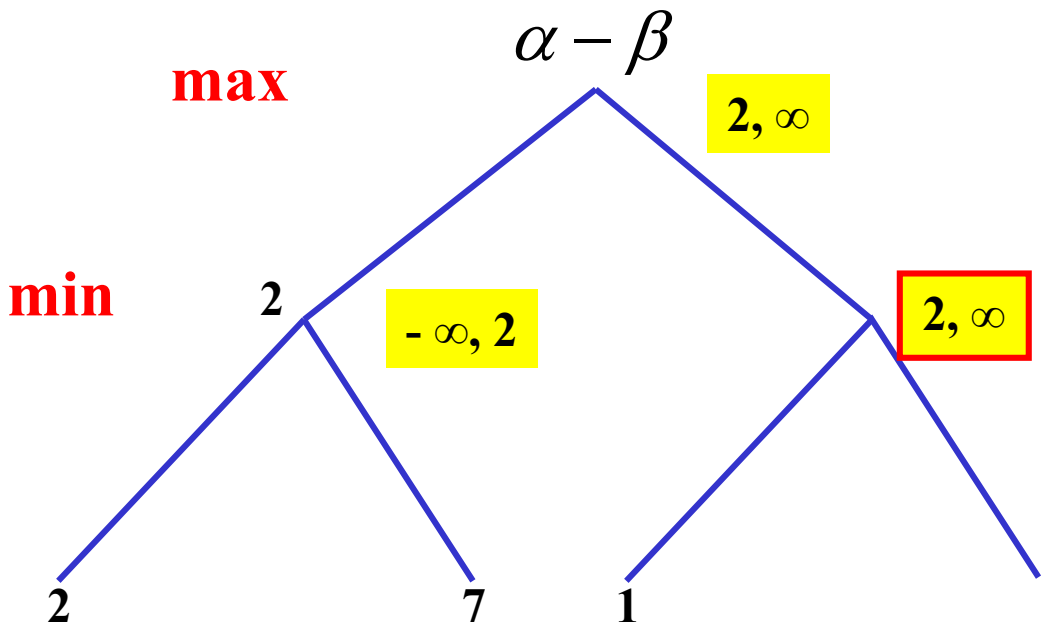
MIN·VALUE now returns 2 to its caller.

Alpha-Beta Pruning in action



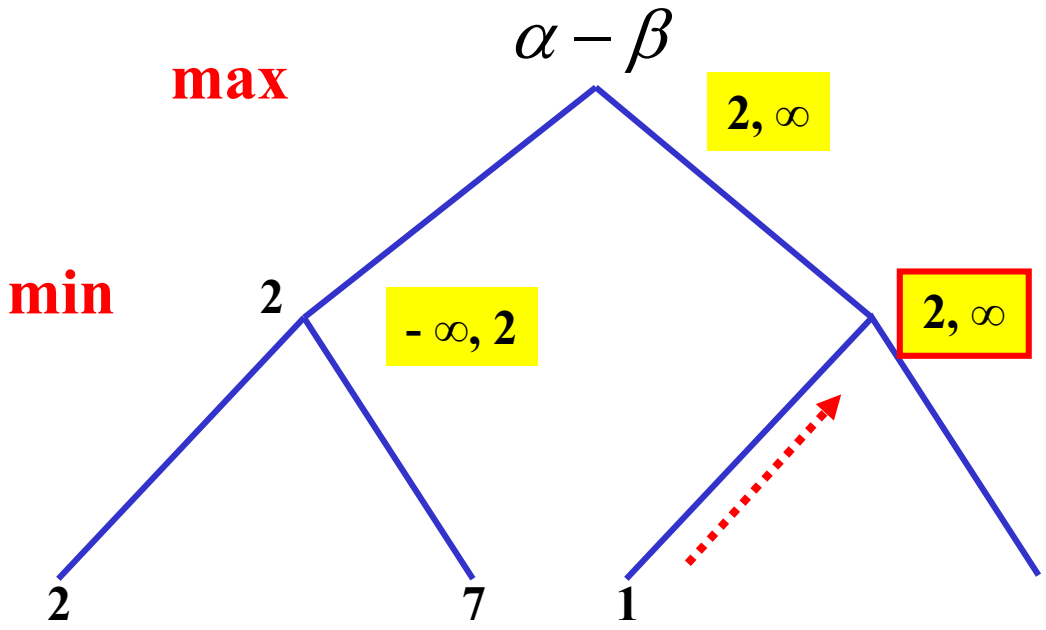
*The calling **MAX·VALUE** now sets **α** to 2, since it is bigger than -∞. Note that the range of [alpha-beta] says that the score will be greater or equal to 2 (and less than ∞).*

Alpha-Beta Pruning in action



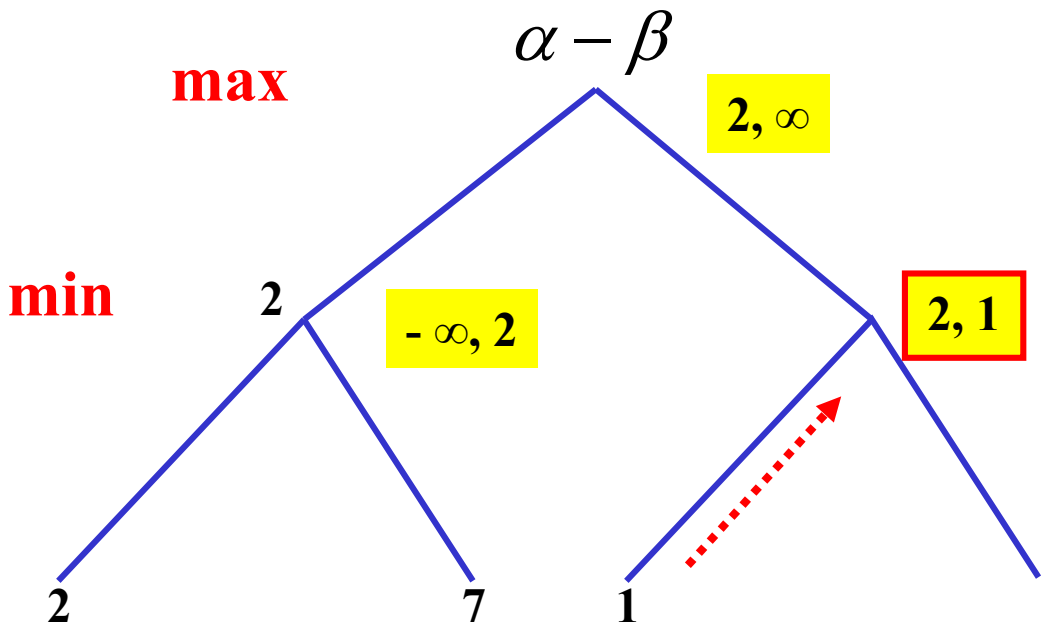
MAX·VALUE now calls MIN·VALUE with an updated range of [alpha-beta].

Alpha-Beta Pruning in action



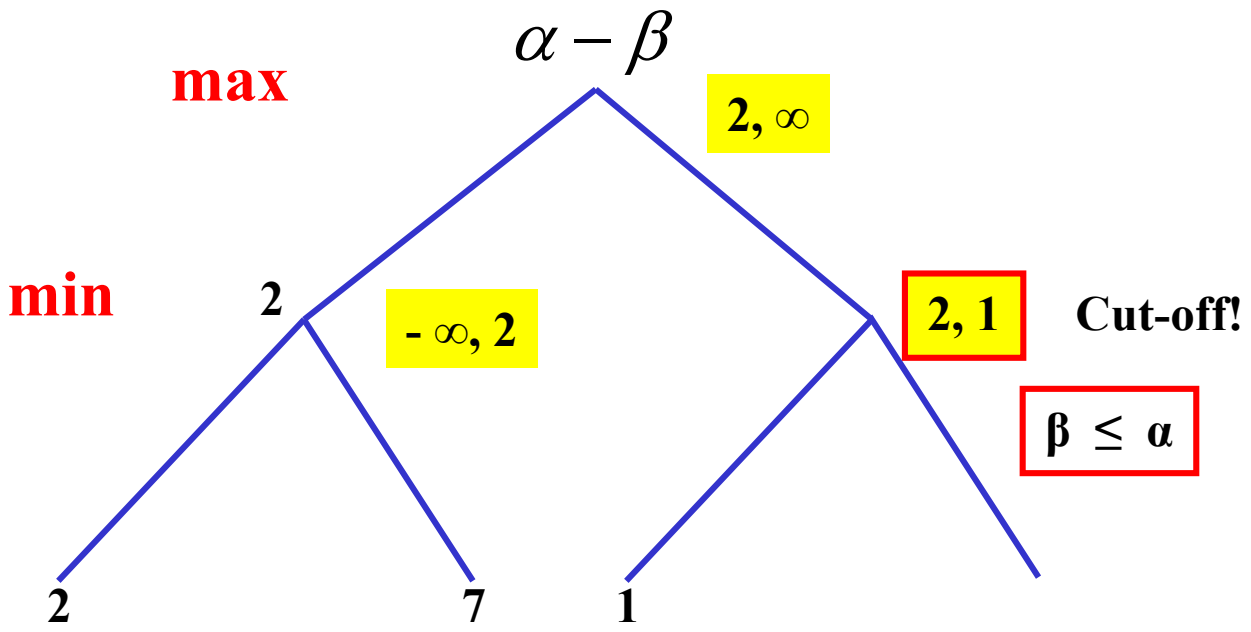
MIN·VALUE calls MAX·VALUE on the left leaf and it returns a value of 1.

Alpha-Beta Pruning in action



*This is used to update beta in **MIN·VALUE**, since it is less than ∞. Note that at this point, we have a range where **α**=2 is greater than **β**=1.*

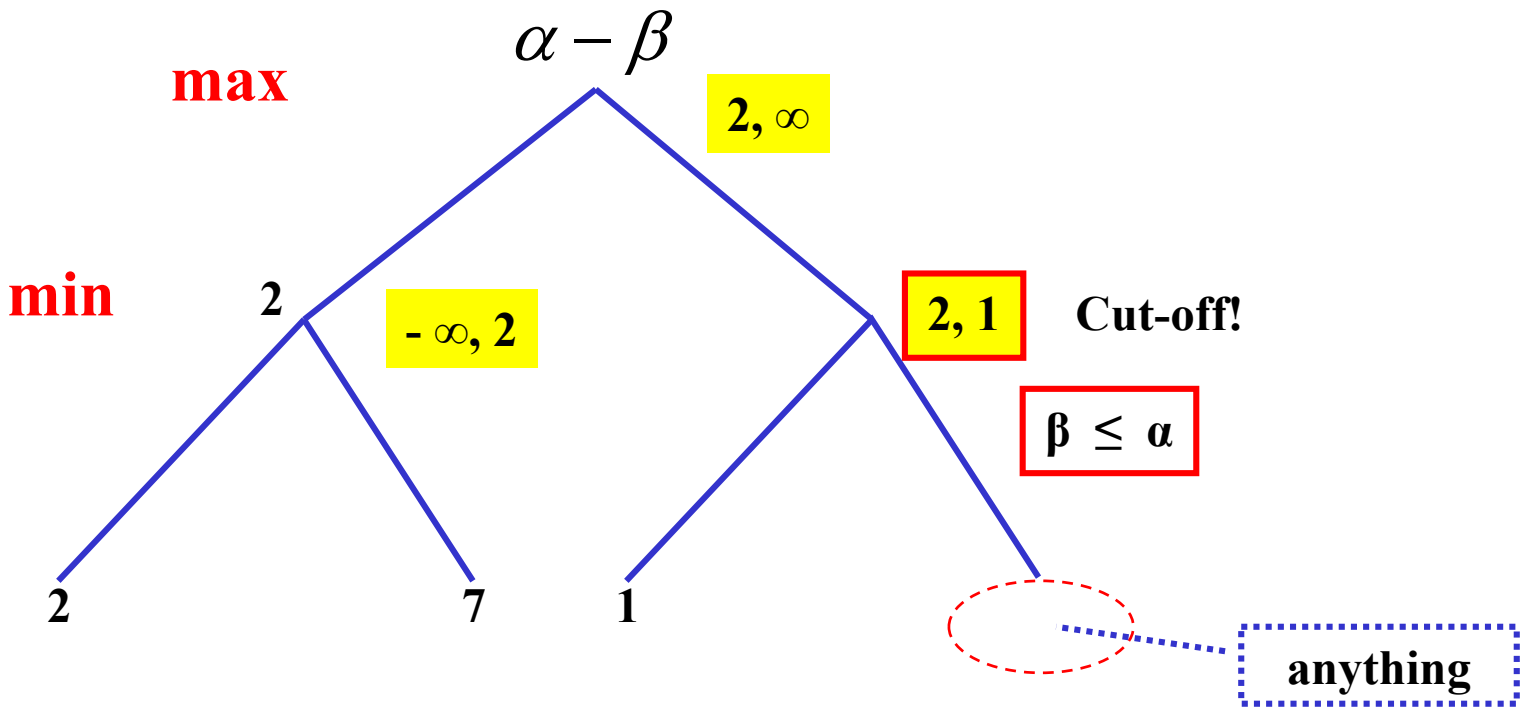
Alpha-Beta Pruning in action



*This is used to update beta in **MIN·VALUE**, since it is less than ∞. Note that at this point, we have a range where **α**=2 is greater than **β**=1.*

*This situation signals a cut-off in **MIN·VALUE** and it returns beta(=1), without looking at the right leaf.*

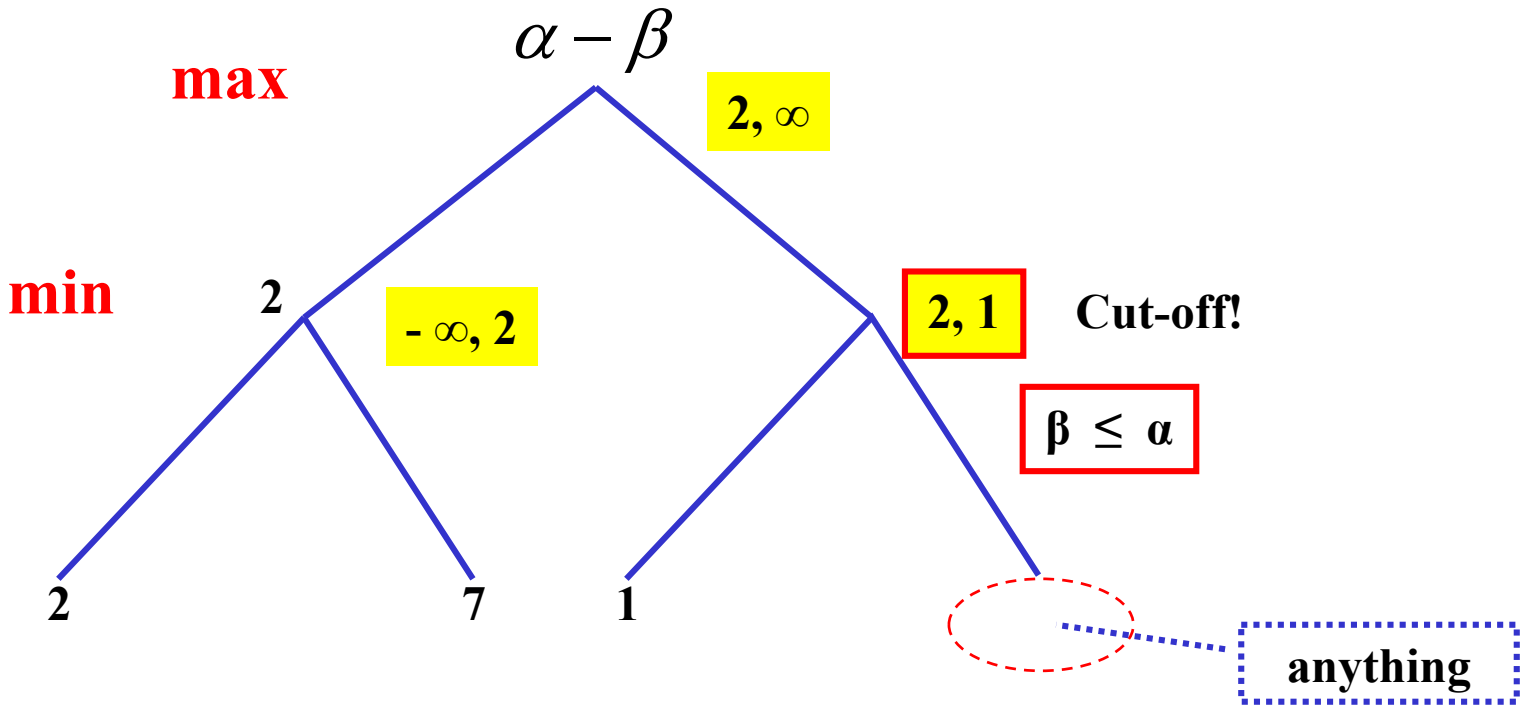
Alpha-Beta Pruning in action



*This situation signals a cut-off in **MIN·VALUE** and it returns beta(=1), <u>without looking at the right leaf</u>.*

*So, basically we had already found a move that guaranteed us a score ≥ **2** so that when we got into a situation where the score was guaranteed to be ≤ **1**, we could stop.*

Alpha-Beta Pruning in action



*So, a total of **3** static evaluations were needed instead of the **4** we would have needed under pure **Min·Max.***

α-β (NegaMax form) *Alpha-Beta Pruning in a more compact form*

α – is the best score for MAX; **β** – is the best score for MIN
Initial call is ***ALPHA·BETA****(state, -∞, ∞, MAX·DEPTH)*

*function **ALPHA·BETA**(state, α, β, depth)*
- *if (depth == 0) then return EVAL(state)*
- *For each **s** in SUCCESSORS(state) do*
  - ***α** = MAX(α, -**ALPHA·BETA**(s, -β, -α, depth-1))*
  - *If(α ≥ β) Then return **α** //cut-off*
- *end*

return α

Basically, this exploits the idea that minimizing is the same as maximising the negatives of the scores.

α-β (NegaMax form) *Alpha-Beta Pruning in a more compact form*

α – is the best score for MAX; **β** – is the best score for MIN
Initial call is ***ALPHA·BETA****(state, -∞, ∞, MAX·DEPTH)*

Modified version

function ***ALPHA·BETA****(state, α, β, depth,* ***MAXPLY****)*

- *if ((depth == 0) and (**MAXPLY is odd**))*
  then return ***-EVAL(state)***

  *else if ((depth == 0) and (**MAXPLY is even**)) return* ***EVAL(state)***
- *For each* ***s*** *in SUCCESSORS(state) do*
  α *= MAX(α, -****ALPHA·BETA****(s, -β, -α, depth-1,* ***MAXPLY****))*
  If(α ≥ β) Then return ***α*** *//cut-off*

  end

return α

Basically, this exploits the idea that minimizing is the same as maximising the negatives of the scores.

α-β (NegaMax form) *Alpha-Beta Pruning in a more compact form*

α – is the best score for MAX; **β** – is the best score for MIN
Initial call is ***ALPHA·BETA****(state, -∞, ∞, MAX·DEPTH, **1**)*

Modified version v2

```
function ALPHA·BETA(state, α, β, depth, colour)
    • if ((depth == 0) then  return (colour * EVAL(state))
    For each s in SUCCESSORS(state) do
        α = MAX(α, -ALPHA·BETA(s, -β, -α, depth-1, -colour))
        If(α ≥ β) Then return α  //cut-off
      end
    return α
```

Basically, this exploits the idea that minimizing is the same as maximising the negatives of the scores.

Key points about α-β

1. Guaranteed same value as **Max-Min**.

2. In a perfectly ordered tree, expected work is $O(b^{d/2})$ vs. $O(b^d)$ for **Max-Min**, so can search twice as deep with the same effort!

3. With good move ordering, the actual running time is close to optimistic estimate.

Game Program

1. Move generator (heuristically ordered moves)	50%
2. Static evaluation	40%
3. Search control	10%

In practice,

- **Openings**
- **End games**

} Played by looking up moves in a **Database**

[all in place by late 60's]

Move Generator

1. Legal moves

2. Ordered by
   - most valuable victim
   - least valuable agressor

3. Killer heuristic

Static Evaluation

Initially	Very complex
70's	Very simple (material)
1997 (IBM Deep Blue)	• Deep searches: moderately complex (hardware) • PC programs: elaborate, hand-tuned

Practical matters

* It is not wise to fix the max. depth of the search.

Variable branching

Iterative Deepening

- Order best move from last search first
- use previous backed up value to initialise [α, β]
- keep track of repeated positions (transposition tables)

Horizon Effect

- quiescence
- pushing the inevitable over search horizon

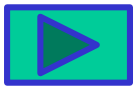
Parallelisation

- hard to do effectively

http://chessprogramming.wikispaces.com/Transposition+Table

Other Games

Backgammon



- Involves **randomness** – dice rolls
- machine-learning based player was able to draw the world champion

Bridge

- Involves **hidden information** – other player's cards, and **communication** during bidding
- Computer players play well but do not bid well

Go

World champion – AlphaGo (2016)

Note, Chess: (b≈35, d≈80)

- No new elements but huge branching factor (b≈250, d≈150); exhaustive search is infeasible
- difficult to evaluate board positions and moves

Other Games

AlphaGo

- Humans make a move based on whether a move "feels right"

- precise search tree-based computation

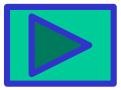
- uses Monte Carlo Tree Search + 2 Deep Neural Networks + reinforcement learning

2 Deep Neural Networks

1. **Value network** = evaluate board positions;
   reduces depth of search
2. **Policy network** = sample actions (select moves)
   = predict the outcome at state s (prunes subtree); reduces breadth of search

Other Games

AlphaGoZero



• trains solely from self-play RL, without any supervision, without human data

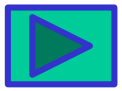
• input features: only black and white stones

• uses Monte Carlo Tree Search without rollouts + 1 Deep Neural Network + new reinforcement learning that incorporates lookahead search inside the training loop

Only 1 Deep Neural Network with the following outputs

1. **Value** = probability of winning at a board position;
2. **Policy** = move probabilities (probability of selecting each move)
   = predict the outcome at state s (prunes subtree); reduces breadth of search

AlphaZero



Games

Observations

- Initially, computers excel in well-defined activities where rules are clear
  - chess
  - mathematics
- Now, the focus is more on less defined activities (vision, language and common sense) and problems that cannot be solved by brute force.
- Success comes after a long period of gradual refinement.

For more details on building game programs, visit:

https://www.ics.uci.edu/~eppstein/180a/links.html